

Slide 1 - Title

Detailed explanation of the proposed solution

- Proxy layer sitting between every agent and the tools/agents it calls — observes real behavior instead of trusting the agent's self-report
- Each agent registers a contract at startup: allowed tools, declared goal, output format, rate limits
- Every action is intercepted, checked against the contract, scored for drift, and logged before it's allowed through

How it addresses the problem

- Verification — checks every action against the agent's declared scope in real time
- Detection — compares current behavior to the agent's own rolling baseline, catching drift as it happens
- Enforcement — graduated response (flag, throttle, halt) stops a misbehaving agent before damage is done

Innovation and uniqueness of the solution

- Framework-agnostic — works across LangChain, CrewAI, AutoGen, or custom agents as a thin proxy
- The audit log itself is the product — exportable as an integrity report, useful beyond just security
- Implements independent, out-of-band monitoring — now a named requirement in global AI safety guidance, not an optional extra

Slide 2: Technical Approach

Technologies to be used

- Backend proxy: FastAPI (Python) or Express (Node.js)
- Rules and contracts: JSON schema per agent
- Drift detection: rolling statistical baseline using z-score/EWMA
- Audit trail: SQLite + hash chaining (hashlib)
- Dashboard: React or lightweight HTML frontend

Methodology and process for implementation

- Agent registers contract → action intercepted by proxy → contract compliance check → drift score calculated against agent's baseline → action hashed into audit chain → approved actions pass through, violations escalate up the response ladder
- Demo agents (2-3) are scripted so one deliberately misbehaves, letting the flag → throttle → halt sequence run live

V

visualize

v

visualize show_widget

Slide 3: Workflow / Wireframe / Use Case

Step-by-step workflow

- Agents A and B each carry their own contract and route every action through Twilight
- Twilight runs the contract check and drift scoring on each action before it's allowed anywhere
- Approved actions pass through to tools/other agents; everything checked (pass or fail) gets logged into the audit chain
- The dashboard pulls live from that chain, so what it shows can't be faked by the agent itself

Represent the system visually

- Two agents feed into a central Twilight block (contract check, drift scorer, hash log, response ladder as sub-modules) which gates access to tools/other agents on one side and feeds the dashboard on the other

Key components and interactions

- Agent Contract — defines the boundary an agent is allowed to operate in
- Twilight proxy — the enforcement point every action must pass through
- Audit chain — the single source of truth the dashboard reads from, independent of the agents

v

visualize

v

visualize show_widget

Slide 4: Feasibility and Viability

Feasibility analysis

- Entire prototype is buildable with mainstream, well-documented tools — no exotic infra, no GPU, no trained model
- Drift detection uses simple statistics (z-score/EWMA) instead of ML, keeping the build realistic for the timeframe
- Contract checks run in constant time, so the proxy doesn't become the bottleneck of the demo

Potential challenges and risks

- Agent legitimately changes its goal mid-session, risking a false positive
- Adding a proxy layer on every action introduces latency

- An agent could try to bypass the proxy and call tools directly

Strategies for overcoming these challenges

- Track baselines per declared goal, not globally, so legitimate goal shifts don't trigger false flags
 - Keep every check $O(1)$ — allow-list plus lightweight stats — matching sub-millisecond precedent from production policy engines
 - Enforce that all tool access must route through Twilight's gateway/API key, closing off any direct bypass path
-

Slide 5: Research, References and Future Scope

Research and References

- MI9: An Integrated Runtime Governance Framework for Agentic AI — arXiv 2508.03858 (2025)
- OWASP Agentic AI Top 10 (2024–26)
- Microsoft Agent Governance Toolkit — GitHub (2026)
- Characterizing Faults in Agentic AI — arXiv 2603.06847 (2026)
- The 2026 Singapore Consensus on Global AI Safety Research Priorities
- Google Agent2Agent (A2A) Protocol

Future Scope

- Extend drift detection from statistical scoring to a learned anomaly model as more real agent traffic data becomes available
- Add support for hardware-attested audit trails (TEE-backed) for enterprise-grade tamper-proofing
- Integrate with agent identity standards like A2A's signed Agent Cards for stronger cross-network trust
- Build multi-tenant dashboards so an organization can monitor several independent agent networks from one view